



PARSEFLOW – AI BASED DOCUMENT CLASSIFIER AND ORGANISER

**FULL STACK
DEVELOPMENT
PROJECT
MAY 2026**

FULL STACK DEVELOPMENT PROJECT REPORT SUBMITTED IN
PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD
OF THE DEGREE OF **BACHELOR OF TECHNOLOGY IN ARTIFICIAL
INTELLIGENCE AND DATA SCIENCE** OF THE ANNA UNIVERSITY

**PROJECT
WORK**

**BATCH
2024 – 2028**

Submitted by
YUGENDRA N -722824243255

Under the Guidance of
Mr. S. Sam Peter M.E., (Ph.D.)
Assistant Professor

Department of Artificial Intelligence and Data Science
Sri Eshwar College of Engineering
(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

Sri Eshwar College of Engineering
(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

BONAFIDE CERTIFICATE

Certified that this Report titled “**PARSEFLOW – AI BASED DOCUMENT CLASSIFIER AND ANALYSER**” is the bonafide work of **YUGENDRA N – 722824243255** who carried out the project work under my supervision.

SIGNATURE

Dr.M.Mohammed Mustafa M.E.,MBA., Ph.D.

HEAD OF THE DEPARTMENT

Associate Professor,
Department of Artificial Intelligence
and Data Science,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

SIGNATURE

Mr. S. Sam Peter M.E., (Ph.D.)

SUPERVISOR

Assistant Professor,
Department of Artificial Intelligence
and Data Science,
Sri Eshwar College of Engineering,
Coimbatore – 641 202.

Submitted for the Autonomous End Semester Project Viva-Voce held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

YUGENDRA N [722824243255]

To declare that the project entitled “ **PARSEFLOW – AI BASED DOCUMENT CLASSIFIER AND ANALYSER** ” submitted in partial fulfilment to the University as the project work of Bachelor of Technology (Artificial Intelligence and Data Science) Degree, is a record of original work done by us under the supervision and guidance of **Mr. S. Sam Peter, M.E., (Ph.D.)** Assistant Professor , Department of Artificial Intelligence And Data Science, Sri Eshwar College of Engineering, Coimbatore.

Place: Coimbatore

Date:

[YUGENDRA N]

Project Guided by,

Mr. S. Sam Peter, M.E., (Ph.D.)

Assistant Professor / AI&DS

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. I take this opportunity to express my gratitude and thanks to everyone who helped me in my project. I would like to thank the management for the constant support provided by them to complete this project.

It is indeed our great honor bounded duty to thank our beloved Chairman **Mr. R. Mohanram**, for his academic interest shown towards the students. We are indebted to our Director **Mr. R. Rajaram**, for motivating and providing us with all facilities.

I wish to express my sincere regards and deep sense of gratitude to **Dr. Sudha Mohanram, M.E, Ph.D. Principal**, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr.M.Mohammed Mustafa M.E., MBA., Ph.D.** Head of Artificial Intelligence and Data Science Department for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department.

I express my thanks to my project Coordinator **Mr. S. Sam Peter, M.E., (Ph.D.)**, Assistant Professor of Artificial Intelligence and Data Science Department for the valuable guidance and encouragement given, to us for this project.

I solemnly express our thanks to all the teaching and nonteaching staff of the Artificial Intelligence and Data Science Department, family and friends for their valuable support which inspired us to work on this project.

COLLECTION OF CONTENT

CONTENT	PAGE NO
LIST OF FIGURES	1
ABSTRACT	2
1. INTRODUCTION	
1.1. OVERVIEW	3
1.2. OBJECTIVE	3
1.3. SCOPE	4
1.4. PROBLEM STATEMENT	4
2. SYSTEM ANALYSIS	
2.1. EXISTING SYSTEM	5
2.2. LIMITATIONS OF EXISTING SYSTEM	5
2.3. PROPOSED SYSTEM	6
2.4. ADVANTAGES OF PROPOSED SYSTEM	6
3. SYSTEM DESIGN	
3.1. ARCHITECTURE OVERVIEW	7
3.2. COLLECTION ARCHITECTURE	7
3.3. DATA FLOW DIAGRAM (DFD)	8
3.4. ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM)	9
4. DATABASE DESIGN	
4.1. RELATIONAL MODELING	10
4.2. COLLECTION STRUCTURE	10
4.3. PRIMARY KEYS AND REFERENCES	11
4.4. NORMALIZATION	11
5. ADVANCED MONGODB FEATURES	
5.1. DOCUMENT BASED STORAGE	13
5.2. EMBEDDING AND REFERENCING	13
5.3. USE CASE	14
6. MONGODB IMPLEMENTATION	
6.1. COLLECTION CREATION QUERIES	15
6.2. DATA INSERTION QUERIES	16

6.3. JOINS	16
6.4. ADVANCED QUERIES	16
6.5. AGGREGATONS	16
7. QUERY OPTIMIZATION AND PERFORMANCE	
7.1. INDEXING STRATEGIES	18
7.2. PERFORMANCE TUNING	18
7.3. EXPLAIN ANALYZE USAGE	19
8. TRANSACTION MANAGEMENT	
8.1 ACID PROPERTIES	20
8.2 TRANSACTION HANDLING	20
8.3 MVCC (CONCURRENCY CONTROL)	21
9. RESULTS AND ANALYSIS	
9.1 SAMPLE DATA OUTPUT	22
9.2 REPORTS GENERATED	22
9.3 PERFORMANCE OBSERVATIONS	23
10. APPLICATION INTEGRATION	
10.1 BACKEND INTEGRATION	24
10.2 FRONTEND INTEGRATION	24
10.3 DATABASE CONNECTIVITY	24
11. CONCLUSION	
11.1 SUMMARY OF WORK	26
11.2 LEARNING OUTCOMES	26
11.3 FUTURE ENHANCEMENTS	27
12. REFERENCES	
A. BOOKS	28
B. WEBSITES	28
C. DOCUMENTATION	28
13. APPENDIX	
A. SCRIPTS	29
B. SAMPLE OUTPUTS	30
C. SCREENSHOTS	37

LIST OF FIGURES

FIGURE NO	LIST OF FIGURES	PAGE NO
3.1	SYSTEM ARCHITECTURE	9
3.2	MONGODB ARCHITECTURE	10
3.3	DATA FLOW DIAGRAM	11
3.4	ER DIAGRAM	12
13.1	HOME PAGE	38
13.2	LOGIN PAGE	38
13.3	DASHBOARD PAGE	39
13.4	DOCUMENTS LIBRARY PAGE	39
13.5	UPLOAD DOCUMENTS PAGE	40
13.6	EXPORT PAGE	40

ABSTRACT

The ParseFlow project is a full-stack web-based application designed to automate document classification, organization, and understanding using AI-driven technologies. The system is developed using React.js for the frontend, Node.js and Express.js for the backend, and Collection as a NoSQL database for efficient storage of document metadata. The main objective is to reduce manual effort by automatically analyzing and structuring documents based on their content.

ParseFlow supports various document types such as identity proofs, financial records, invoices, and tax-related files. Each document is processed through a pipeline that includes preprocessing, machine learning-based classification, OCR-based text extraction, and language model analysis. Based on this process, the system determines the document type, extracts key metadata, and assigns it to appropriate categories i.e Identity, Financial, or Tax.

The backend manages authentication, API handling, and decision logic while storing metadata such as document type, category, confidence score, and file paths in Collection. The frontend provides a responsive interface for uploading documents, viewing categorized results, and interacting with an intelligent assistant (DocBot) to query stored data. This also involves an ML processing layer for identifying the type of document.

The project also incorporates modular architecture and scalable processing layers. Machine learning and OCR components operate as independent services, while the backend acts as an orchestration layer. Security features such as user-level data isolation, encrypted storage, and controlled file access ensure data privacy and integrity.

INTRODUCTION

1.1 OVERVIEW

The project titled **ParseFlow** is a full-stack web application designed to automate the classification, organization, and understanding of documents using AI-driven technologies. The system is built using **React.js** for the frontend, **Node.js** and **Express.js** for the backend, and **Collection** for data storage, integrating machine learning, OCR, and language models to enable intelligent document processing.

Parseflow supports multiple types of documents such as identity proofs, financial records, invoices, and tax-related files. Each document is processed through a structured pipeline that includes machine learning-based classification, OCR-driven text extraction, and language model analysis. This enables the system to automatically identify document types, extract key information, and organize files into appropriate categories without requiring user intervention.

1.2 OBJECTIVE

The primary objectives of the ParseFlow system are:

- To provide an intelligent platform for automated document classification and organization.
- To extract meaningful information from documents using ML, OCR, and language models.
- To enable users to interact with their documents
- To automatically categorize documents into structured storage without user intervention.
- To demonstrate the use of **advanced database concepts and full-stack development.**

1.3 SCOPE

The scope of this project includes:

- Automated classification and organization of various document types.
- Development of a **web-based interactive system** accessible through browsers.
- Implementation of:
 - User authentication (JWT-based)
 - Document metadata storage using Collection
 - RESTful API services
 - AI pipeline integrating ML classification, OCR, and LLM analysis
- Structured storage of documents based on category and document type
- Interaction with stored documents using an intelligent assistant (DocBot)

However, the project is limited to:

- Dependence on model accuracy for document classification
- Use of AI-based processing without real-time external data integrations

1.4 PROBLEM STATEMENT

Understanding and managing documents is often difficult due to:

- Lack of proper organization and structured storage
- Manual effort required to sort and categorize files
- Absence of intelligent systems to interpret document content
- Difficulty in locating important documents quickly

Traditional systems store documents in static formats (folders or file systems), which do not effectively provide:

- Automatic classification of documents
- Contextual understanding of document content
- Interaction or guidance related to stored documents

Therefore, there is a need for a system that:

- Provides **automated and intelligent document organization**
- Reduces manual effort in document management
- Allows users to interact with their documents effectively

ParseFlow addresses this gap by offering an AI-powered platform that transforms unstructured documents into organized, searchable, and interactive data.

2.1 EXISTING SYSTEM

In the current scenario, government documents are primarily presented through:

- Cloud storage platforms
- Local file systems and manual folder structures
- Email attachments and document sharing tools
- Basic document management systems used in organizations

These systems mainly focus on storing and retrieving files. While they provide accessibility and storage, they lack intelligent features that can understand and organize documents automatically.

Users typically need to:

- Manually upload and organize documents into folders
- Remember file names or locations for retrieval
- Spend time searching for relevant documents

There is no unified system that combines **automated classification, content understanding, and intelligent interaction** within a single platform.

2.2 LIMITATIONS OF EXISTING SYSTEM

The existing systems have several limitations:

- **Lack of Intelligence**
Documents are stored as files without understanding their content.
- **No Automated Classification**
Systems do not automatically identify document types.

- **No Contextual Understanding**

Existing systems cannot extract or interpret key information from documents.

- **Fragmented Storage**

Documents are scattered across multiple platforms, making management difficult.

2.3 PROPOSED SYSTEM

The proposed system, **ParseFlow**, is an AI-powered web-based platform that introduces intelligent automation into document management.

Key features of the proposed system include:

- **Automated Document Classification**

Uses a machine learning model to identify document types quickly.

- **Fallback Processing Pipeline**

If ML confidence is low, the system uses OCR and language models for deeper analysis.

- **Interactive Timeline Control**

Users can control the categories timeline manually or automatically.

- **Real-Time Impact Metrics**

Displays economic, social, and environmental indicators dynamically.

- **Intelligent Assistant (DocBot)**

Allows users to interact with their stored documents and retrieve information.

- **Full-Stack Architecture**

Built using:

- Frontend: React.js
- Backend: Node.js & Express
- Database: Collection

This system transforms document analysing and understanding into a **dynamic and immersive experience**.

2.4 ADVANTAGES OF PROPOSED SYSTEM

The ParseFlow platform offers several advantages over traditional systems:

- **Automated Organization**

Sections provide a realistic representation of category implementation.

- **Interactive Learning Experience**

Users can actively engage with DocBot

- **Better Understanding of documents**

Shows data of the document whenever needed

- **Time-Based Analysis**

Demonstrates how documents are analysed and stored.

- **Centralized System**

All features are available in one unified platform.

- **User-Friendly Interface**

Intuitive design improves accessibility and usability.

- **Educational Value**

Useful for students, researchers, and businesses.

3.1 ARCHITECTURE OVERVIEW (CLIENT–SERVER MODEL)

The **ParseFlow** system follows a client–server architecture where users interact with a React-based frontend to upload, view, and manage documents. The backend, built using Node.js and Express, handles API requests, authentication, and decision-making logic while communicating with the Collection database for storing document metadata and user-specific information. Additionally, the core processing pipeline integrates machine learning for document classification, OCR for text extraction, and language models for content understanding, enabling automated categorization and analysis

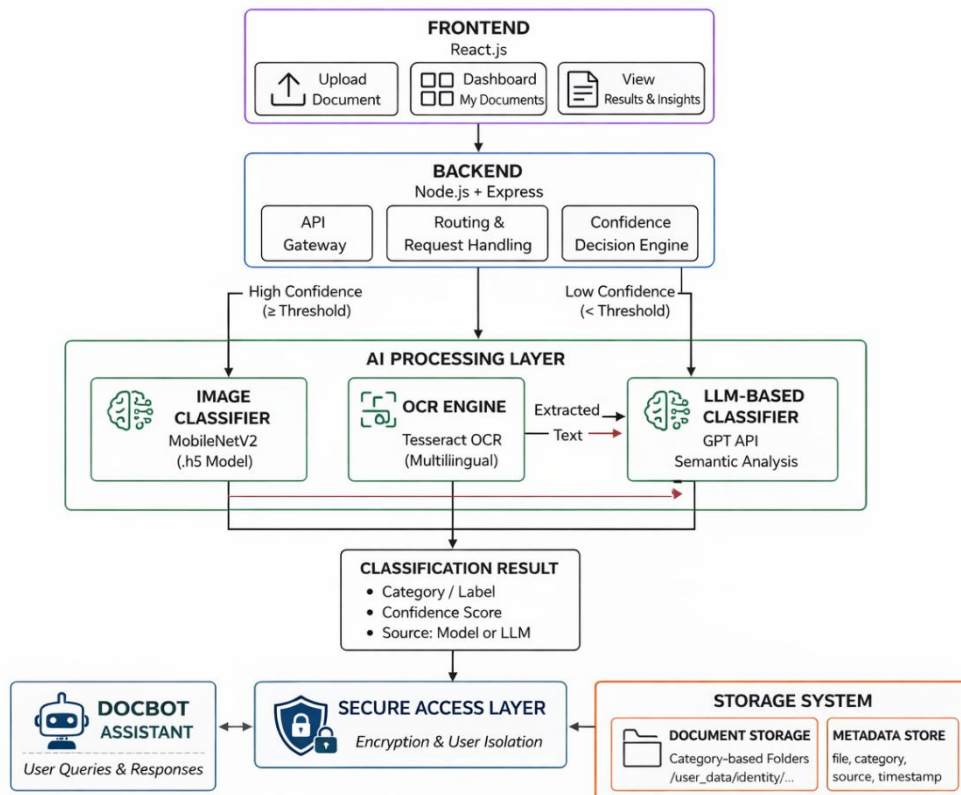


Fig 3.1 System Architecture

3.2 COLLECTION ARCHITECTURE

The **ParseFlow** system follows a client–server architecture where users interact with a React-based frontend to upload and manage documents. The backend, built using Node.js and Express, handles API requests, authentication, and processing logic while communicating with the Collection database. Collection efficiently manages document-oriented data such as user information, document metadata, classification results, and storage references using flexible JSON-like structures. This approach enables scalable storage, faster retrieval, and efficient handling of both structured and unstructured document data.

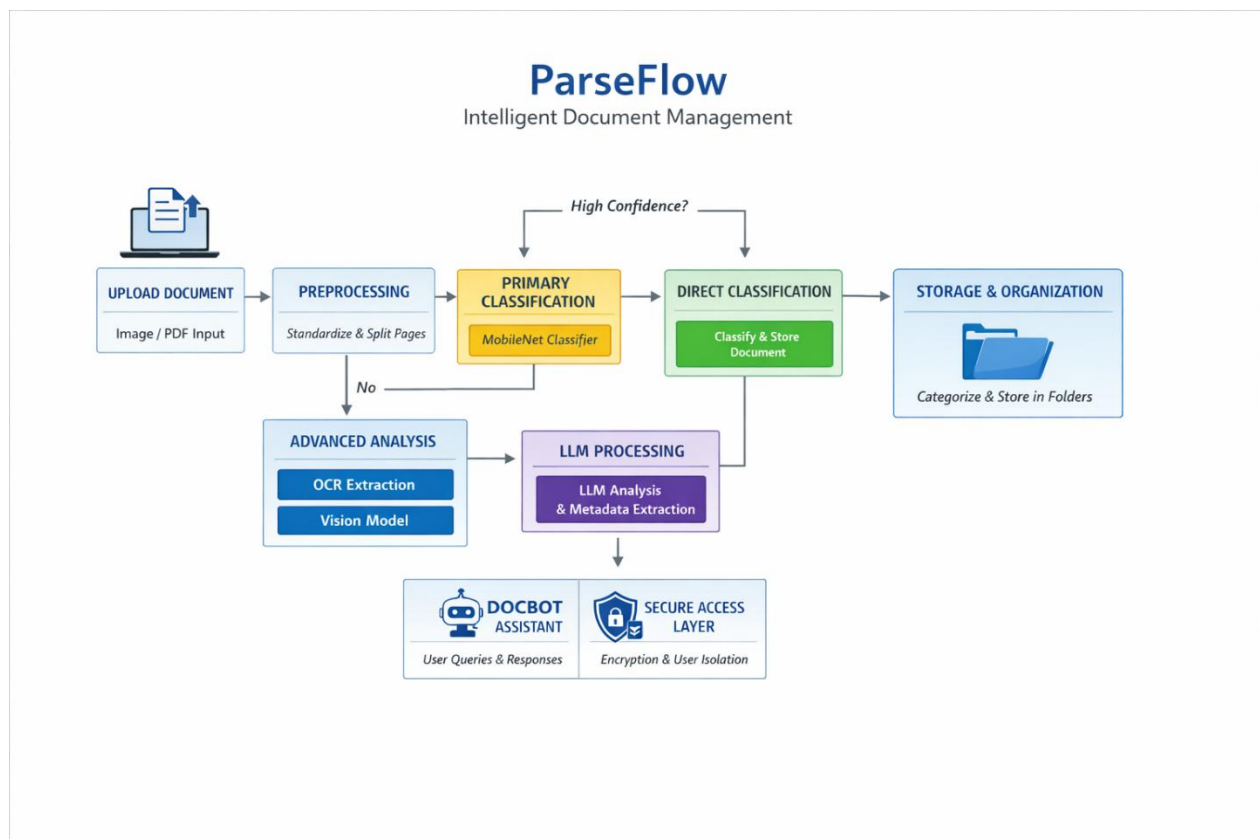


Fig 3.2 Collection Architecture

3.3 DATA FLOW DIAGRAM (DFD)

The **ParseFlow** system processes user requests through the React frontend, where users upload and manage documents. The frontend sends data to the Node.js backend via REST APIs, where authentication, request handling, and decision-making logic are performed. The backend first routes the document through a machine learning model for classification; if confidence is low, it triggers OCR-based text extraction followed by language model analysis for deeper understanding. The processed results, including document type, category, and metadata, are then stored in the Collection database. Finally, the categorized documents and insights are returned to the frontend, where users can view organized files and interact with them through the DocBot interface..

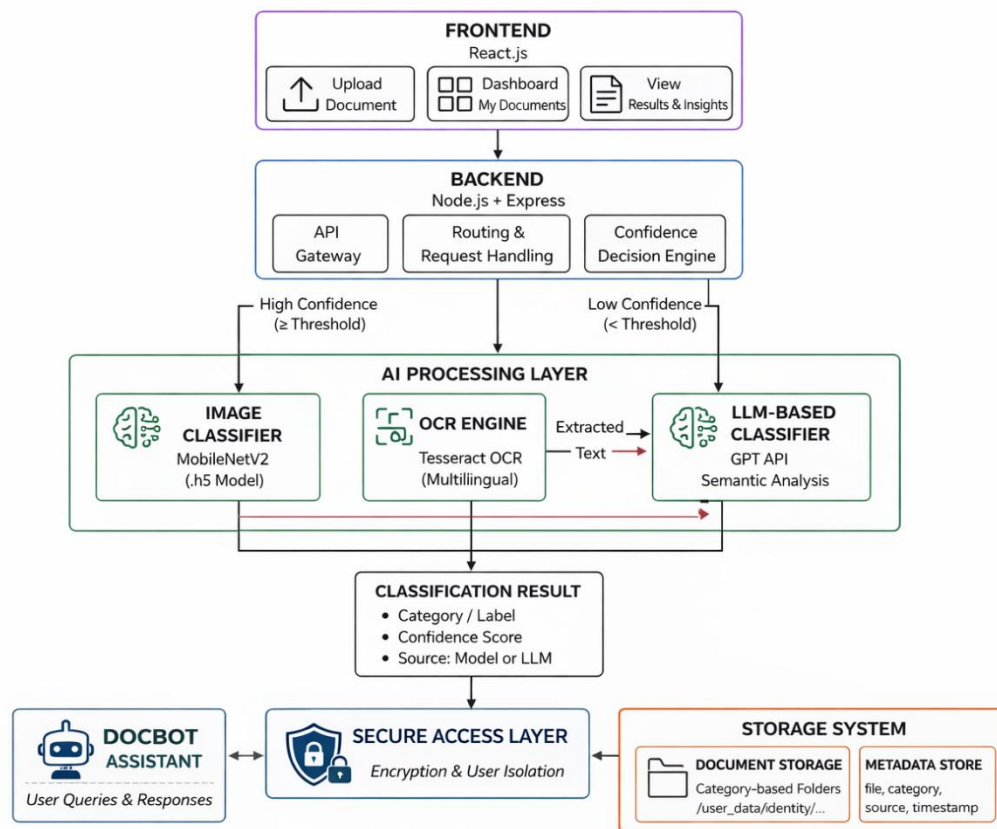


Fig 3.3 Data Flow Diagram

3.4 ENTITY RELATIONSHIP DIAGRAM (ER DIAGRAM)

The Entity Relationship (ER) diagram of the ParseFlow system represents the structure and relationships between core entities such as Users, Documents, Categories, and storage areas . Each user can interact with multiple documents, and each document belongs to a specific category, establishing a one-to-many relationship. The entity stores dynamic data related to document type and category, linking users documents and categories. This relational design ensures data integrity, efficient querying, and structured storage within the database.

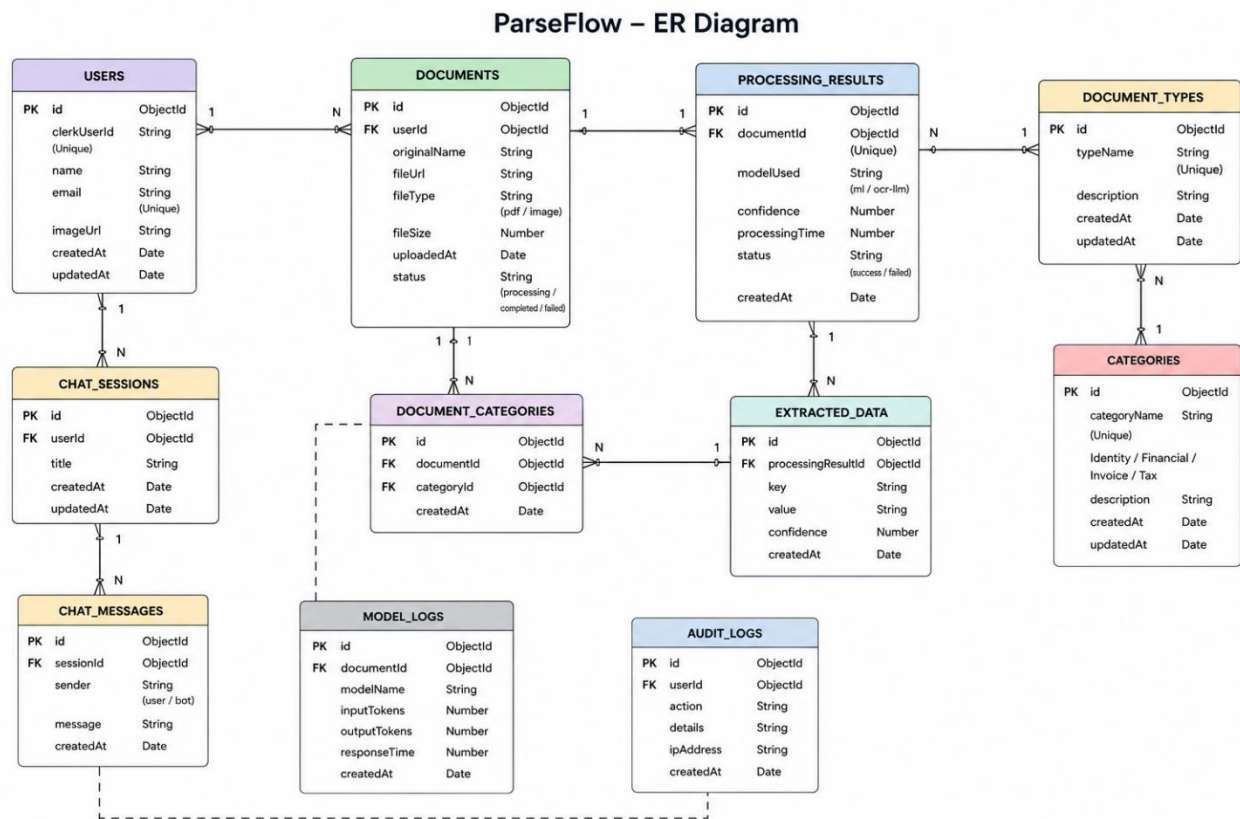


Fig 3.4 ER Diagram

DATABASE DESIGN

4.1 DATA MODELING

The **ParseFlow** system adopts a document-oriented data model implemented using **Collection** to efficiently manage data related to users, documents, classification results, and interactions. In this model, data is stored as flexible JSON-like documents (BSON), where each collection represents a specific entity and each document contains key–value pairs

This data model ensures:

- Flexible schema design for handling both structured and unstructured document data
- Reduced complexity by embedding related information where required
- Efficient data retrieval without complex joins

Relationships between entities are defined using references (such as `userId` and `documentId`), enabling logical connections between collections such as Users, Documents, and Processing Results. This approach allows the system to handle dynamic data, maintain consistency, and scale efficiently as the number of users and documents increases.

4.2 COLLECTION STRUCTURE

The ParseFlow database consists of several core collections designed to represent different components of the system:

Users Collection

This Collection stores all registered user information:

- `user_id` (Primary Key)
- `name`
- `email` (Unique)
- `password`
- `role` (admin/user)

- created_at

It is responsible for authentication and user management.

Documents Collection

This Collection stores detailed information about documents:

- doc_id (Primary Key)
- title
- description
- category_id (Foreign Key)
- created_at

Each document is linked to a category.

Categories Collection

This Collection classifies documents into different domains:

- category_id (Primary Key)
- name
- description

Examples include tax documents , bills , etc.

Storage Structure

Documents are physically stored in a structured directory:

- userId / category / document_type / file

This ensures organized and scalable file management.

Additional Supporting Collections

- **Chat Sessions** → Stores DocBot interactions
- **Chat Messages** → Stores user and bot messages
- **Logs** → Tracks processing and system activity

4.3 PRIMARY KEYS AND REFERENCES

Primary Keys (PK)

- Unique identifier for each record
- Cannot be null or duplicated
- Examples:
 - user_id in Users Collection
 - doc_id in documents Collection
 - category_id in categories Collection

References

- Establish relationships between Collections
- Relationships are maintained using fields like userId and documentId
- Collection uses references instead of strict foreign keys

Key Relationships:

- Documents.userId → Users._id
- Categories.user_id → Users.user_id
- ChatSessions.userId → Users._id
- Processing.documentId → Documents._id

These relationships help maintain consistency and allow flexible schema design.

4.4 NORMALIZATION

Unlike relational databases, Collection follows a hybrid approach of normalization and denormalization for performance and flexibility.

First Normal Form (1NF)

- All attributes contain atomic (indivisible) values
- No repeating groups or multi-valued attributes
- Example: Each field stores only one value (e.g., one email per user)

Second Normal Form (2NF)

- Achieved after 1NF
- All non-key attributes depend on the entire primary key
- Removes partial dependencies

Third Normal Form (3NF)

- Removes transitive dependencies
- Non-key attributes depend only on the primary key

Benefits of Normalization in ParseFlow

- Faster data retrieval without complex joins
- Improves data consistency
- Enhances query performance
- Simplifies database maintenance

5.1 DOCUMENT BASED DATA

This allows storing semi-structured data in a flexible and efficient format. Unlike traditional relational columns, JSONB enables the storage of complex data such as nested objects and arrays within a single column.

In the ParseFlow system, JSONB is used to store **categories impact data**, which may vary depending on the documents and categories phase..and can be stored within the `impact_data` column.

Advantages:

- Efficient storage
- Faster querying and indexing
- Flexibility to store dynamic and hierarchical data
- Supports advanced operators for filtering and searching

Example Usage:

```
{
  "document_type": "Invoice",
  "category": "Financial",
  "extractedData": {
    "invoice_number": "INV-2026-0847",
    "date": "2026-03-15",
    "amount": 42500
  }
}
```

This allows the system to handle varying categories results without changing the database schema.

5.2 EMBEDDING AND REFERENCING

It allows multiple values to be stored in a single column. This is useful when an entity has a list of related values that do not require a separate Collection.

In the project, arrays can be used for:

- Storing **documents tags or keywords**
- Representing **skills or categories associated with a documents**
- Managing multiple attributes related to a categories

Advantages of Arrays:

- Simplifies schema design for multi-valued attributes
- Reduces the need for additional join Collections
- Allows efficient querying using array operators

5.3 USE CASES IN PROJECT

The ParseFlow system leverages advanced MongoDB features to improve flexibility, performance, and scalability:

- **Dynamic Categories Data Storage**
used to store impact metrics that change over time and differ across documents.
- **Efficient Data Retrieval**
JSONB indexing allows quick access to specific fields within stored JSON data.
- **Flexible Documents Attributes**
Arrays enable storing multiple tags or classifications without complex schema modifications.
- **Reduced Schema Complexity**
Combining relational Collections with minimizes the number of Collections required.

6.1 COLLECTION CREATION QUERIES

The following collections are used to create the core Collections in the ParseFlow database:

// Users Collection

```
{
  _id: ObjectId,
  clerkUserId: String,
  name: String,
  email: String,
  createdAt: Date
}
```

// Documents Collection

```
{
  _id: ObjectId,
  userId: ObjectId,
  filename: String,
  category: String,
  document_type: String,
  confidence: Number,
  filePath: String,
  createdAt: Date
}
```

6.2 Data Insertion Queries

```
db.users.insertOne({  
  name: "Yugen",  
  email: "yugen@example.com",  
  createdAt: new Date()  
});
```

```
db.documents.insertOne({  
  userId: ObjectId("..."),  
  filename: "invoice.png",  
  category: "Financial",  
  document_type: "Invoice",  
  confidence: 0.92,  
  createdAt: new Date()  
});
```

6.3 QUERY OPERATION

Retrieve documents with their usernames:

```
db.documents.find({ userId: ObjectId("...") });
```

Retrieve all by category

```
db.documents.find({ category: "Identity" });
```

6.4 AGGREGATION (REPLACES JOINS)

Retrieve documents:

```
db.documents.aggregate([
  { $match: { userId: ObjectId("...") } },
  { $group: { _id: "$category", count: { $sum: 1 } } }
]);
```

6.5 SUBQUERY EQUIVALENT (\$lookup)

```
db.documents.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "userDetails"
    }
  }
]);
```

QUERY OPTIMIZATION & PERFORMANCE

7.1 INDEXING STRATEGIES

Indexing is one of the most effective techniques used in MongoDB to improve query performance. Indexes allow the database to locate data quickly without scanning the entire Collection.

In the ParseFlow system, indexes are created on frequently queried columns such as:

- `user_id` in the Users and Categories Collections
- `documents_id` in the Documents and Categories Collections
- `category_id` in the Documents Collection

Example Index Creation:

```
CREATE INDEX idx_users_email ON users(email);
```

```
CREATE INDEX idx_documents_category ON documents(category_id);
```

```
CREATE INDEX idx_categories_user ON categories(user_id);
```

Additionally, **GIN (Generalized Inverted Index)** is used for JSONB fields to improve performance when querying categories impact data.

7.2 PERFORMANCE TUNING

Performance tuning involves optimizing queries and database configuration to ensure efficient execution.

In the ParseFlow system, the following techniques are used:

- **Query Optimization**

Writing efficient SQL queries with proper joins and avoiding unnecessary nested queries.

- **Selective Data Retrieval**

Using specific column selection instead of `SELECT *` to reduce data load.

- **Proper Use of Indexes**

Ensuring indexes are applied only where necessary to avoid overhead.

- **Connection Pooling**

Managing database connections efficiently to handle multiple users.

- **Caching Mechanisms** (optional)

Frequently accessed data such as documents lists can be cached to reduce database load.

7.3 EXPLAIN ANALYZE USAGE

MongoDB provides the **EXPLAIN ANALYZE** command to evaluate query performance. It shows how the query is executed and helps identify bottlenecks.

Example:

```
EXPLAIN ANALYZE
```

```
SELECT * FROM categories WHERE user_id = 1;
```

Benefits:

- Displays execution plan of queries
- Identifies slow operations such as sequential scans
- Helps in deciding where indexes are needed
- Improves overall query efficiency

TRANSACTION MANAGEMENT

8.1 ACID PROPERTIES

Transaction management in MongoDB ensures data reliability and consistency through the **ACID properties**:

Atomicity

Atomicity ensures that a transaction is treated as a single unit of work. Either all operations within the transaction are successfully completed, or none are applied

Consistency

Consistency ensures that the database remains in a valid state before and after a transaction. All defined rules, constraints, and relationships must be maintained.

Isolation

Isolation ensures that multiple transactions occurring simultaneously do not interfere with each other. Each transaction operates as if it is executed independently.

Durability

Durability guarantees that once a transaction is committed, the changes are permanently stored in the database, even in case of system failure.

8.2 TRANSACTION HANDLING

MongoDB provides commands to control transactions:

BEGIN

Starts a new transaction.

BEGIN;

COMMIT

Saves all changes made during the transaction permanently.

COMMIT;

ROLLBACK

Reverts all changes made during the transaction if an error occurs.

ROLLBACK;

Example Transaction in ParseFlow

BEGIN;

```
INSERT INTO categories (user_id, documents_id, phase, timeline, impact_data)
VALUES (1, 2, 'Setup Phase', 1, '{"economic":50,"social":40,"environmental":30}');
```

COMMIT;

If any error occurs during this process, a ROLLBACK ensures that no partial data is stored.

8.3 MVCC (Concurrency Control)

MongoDB uses **MVCC (Multi-Version Concurrency Control)** to handle multiple transactions efficiently without locking the entire database.

How MVCC Works:

- Instead of overwriting data, MongoDB creates **multiple versions** of a row.
- Each transaction sees a **consistent snapshot** of the database.
- Readers do not block writers, and writers do not block readers.

Advantages of MVCC:

- Improves performance in multi-user environments
- Reduces locking conflicts
- Ensures high concurrency
- Provides consistent query results

MVCC in ParseFlow

In the ParseFlow system:

- Multiple users can run queries simultaneously
- Data updates do not interrupt other users
- The system maintains smooth and consistent performance even under load

RESULTS AND ANALYSIS

9.1 SAMPLE DATA OUTPUT

The ParseFlow system generates structured outputs based on user interactions and categories inputs. Sample data retrieved from the MongoDB database includes information about users, documents, and categories results.

Example outputs include:

- List of documents with their categories
- Categories records showing phase-wise progression
- Impact metrics

These outputs demonstrate that the system successfully stores, retrieves, and processes data and advanced features.

9.2 REPORTS GENERATED

The system generates analytical reports based on stored data to provide insights into documents performance and usage.

1. Users per Category

This report shows how many users are interacting with documents in each category, helping analyze user engagement.

2. Documents per Category

Displays the number of documents available under each domain such as Education, Healthcare, Environment, etc.

3. Categories Statistics

This report includes:

- Total categories performed
- Most simulated documents
- Phase-wise categories distribution

These reports are generated using queries involving aggregations, and grouping functions.

9.3 PERFORMANCE OBSERVATIONS

The performance of the ParseFlow system was evaluated based on query execution time, data retrieval efficiency, and system responsiveness.

Key Observations:

- **Fast Query Execution**

Indexing on key columns such as `user_id` and `documents_id` significantly improved query speed.

- **Efficient Handling**

flexible storage of categories impact data without affecting performance.

- **Scalability**

The system handled multiple users and categories efficiently due to MongoDB's optimized query engine.

- **Concurrency Support**

MVCC ensured smooth execution of multiple transactions without conflicts.

- **Optimized Joins and Queries**

Proper use of functions improved data retrieval and reporting.

10.1 BACKEND INTEGRATION

The backend of the ParseFlow system is developed using **Node.js and Express.js**, which provides a robust and scalable environment for handling server-side logic. The backend exposes a set of **RESTful APIs** that enable communication between the frontend and the database.

Key backend functionalities include:

- **User Authentication** using JWT (JSON Web Tokens)
- **API Endpoints** for managing users, documents, categories, and storage
- **Business Logic Processing** for categories phases and impact calculations
- **Database Interaction** using SQL queries to communicate with MongoDB

The backend ensures secure data handling, request validation, and efficient processing of user inputs.

10.2 FRONTEND INTERACTION

The frontend of ParseFlow is built using **React.js**, providing a dynamic and responsive user interface. It allows users to interact with the system seamlessly through various components and pages.

Key frontend features include:

- **User Interface Components** such as Dashboard, Documents Library, and Categories View
- **Interactive Visualization** using Three.js for real-time categories rendering
- **API Integration** using Axios to fetch and send data to the backend
- **State Management** for handling user sessions and categories data

The frontend communicates with the backend via HTTP requests and updates the UI dynamically based on the received data.

10.3 DATABASE CONNECTIVITY

The ParseFlow system uses **MongoDB** as its relational database, which is connected to the backend using appropriate database drivers or ORM tools.

Key aspects of database connectivity include:

- Establishing secure connections between Node.js and MongoDB
- Executing queries for data retrieval and storage
- Managing connection pooling for handling multiple users
- Ensuring data integrity through constraints and transactions

The database acts as the central repository for storing users, documents, history and categories data, enabling efficient data access and management.

CONCLUSION

11.1 SUMMARY OF WORK

The ParseFlow project successfully demonstrates the design and implementation of a full-stack application for simulating government documents using modern web technologies and MongoDB. The system integrates a React-based frontend, a Node.js and Express backend, and a MongoDB database to provide a complete and functional platform.

Throughout the project, key database concepts such as relational modeling, normalization, indexing, query optimization, and transaction management were applied effectively. Advanced MongoDB features were utilized to handle dynamic categories data.

Overall, the project achieves its goal of transforming traditional documents analysis into an interactive and data-driven categories experience.

11.2 LEARNING OUTCOMES

During the development of the ParseFlow system, several important concepts and skills were gained:

- Understanding of **MongoDB database design and implementation**
- Knowledge of **query optimization and performance tuning techniques**
- Implementation of **transaction management and ACID properties**
- Experience in **full-stack development using MERN technologies**
- Integration of **frontend, backend, and database systems**

This project provided hands-on experience in building scalable and efficient database-driven applications.

11.3 FUTURE ENHANCEMENTS

Although the ParseFlow system is fully functional, several improvements can be made in the future:

- Integration of **real-time data finding and form filling**.
- Implementation of advanced **AI-based documents analysis and predictions**
- Enhanced **data visualization dashboards and analytics**
- Mobile application support for wider accessibility
- Advanced **user personalization and recommendation system**
- Deployment on cloud platforms for better scalability

These enhancements can further improve the system's usability, performance, and real-world applicability.

REFERENCES

A. BOOKS

- Korth, H. F., Silberschatz, A., & Sudarshan, S. – Database System Concepts, McGraw-Hill
- Elmasri, R., & Navathe, S. – Fundamentals of Database Systems, Pearson
- Rob, P., & Coronel, C. – Database Systems: Design, Implementation, and Management, Cengage Learning
- Kleppmann, M. – Designing Data-Intensive Applications, O'Reilly Media
- McLaughlin, B. – MongoDB: Up and Running, O'Reilly Media

B. WEBSITES

- MongoDB Official Documentation – <https://www.MongoDB.org/docs/>
- Node.js Official Website – <https://nodejs.org/>
- React Official Documentation – <https://react.dev/>
- Express.js Documentation – <https://expressjs.com/>
- Collection (for reference comparison) – <https://www.Collection.com/docs/>
- MDN Web Docs (JavaScript & APIs) – <https://developer.mozilla.org/>
- GeeksforGeeks (Database & SQL concepts) – <https://www.geeksforgeeks.org/>
- W3Schools SQL Tutorial – <https://www.w3schools.com/sql/>

C. DOCUMENTATION

- MongoDB Documentation
- MongoDB Indexing and Performance Tuning Guide
- MongoDB Transaction Documentation
- REST API Design Guidelines (OpenAPI / REST standards)
- Axios Documentation (API communication) – <https://axios-http.com/docs/intro>

A. SQL SCRIPTS

The following SQL scripts were used for database creation, data insertion, and query execution in the ParseFlow system.

1. Collection Creation

```
db.users.insertOne({  
  clerkUserId: "user_12345",  
  name: "Demo User",  
  email: "demo@parseflow.com",  
  createdAt: new Date()  
});
```

```
db.documents.insertOne({  
  userId: ObjectId("USER_ID"),  
  filename: "invoice.png",  
  document_type: "Invoice",  
  category: "Financial",  
  confidence: 0.92,  
  method: "ML",  
  filePath: "/storage/user_123/Financial/Invoice/invoice.png",  
  createdAt: new Date()  
});
```



```
db.processing.insertOne({
  documentId: ObjectId("DOC_ID"),
  document_type: "Invoice",
  category: "Financial",
  confidence: 0.92,
  method: "ML",
  extractedData: {
    invoice_number: "INV-2026-0847",
    date: "2026-03-15",
    amount: 42500
  },
  createdAt: new Date()
});
```

Sample Data Insertion

```
db.documents.find({ userId: ObjectId("USER_ID") });
```

```
db.documents.find({ category: "Identity" });
```

```
db.documents.aggregate([
  { $match: { userId: ObjectId("USER_ID") } },
  { $group: { _id: "$category", count: { $sum: 1 } } }
]);
```

B. SAMPLE OUTPUTS

The system generates structured outputs based on user interactions. Sample outputs include:

- The system correctly classifies uploaded documents into categories such as Identity, Financial, and Tax
- It automatically assigns document types like Aadhaar Card, Invoice, or Driving License
- Extracted metadata such as names, dates, and amounts are stored in structured format
- Documents are organized into user-specific storage folders based on classification results
- The system provides confidence scores and processing methods (ML or LLM) for each document
- DocBot enables users to query and retrieve information from their stored documents

C. SCREENSHOTS

The following screenshots demonstrate the working of the ParseFlow system:

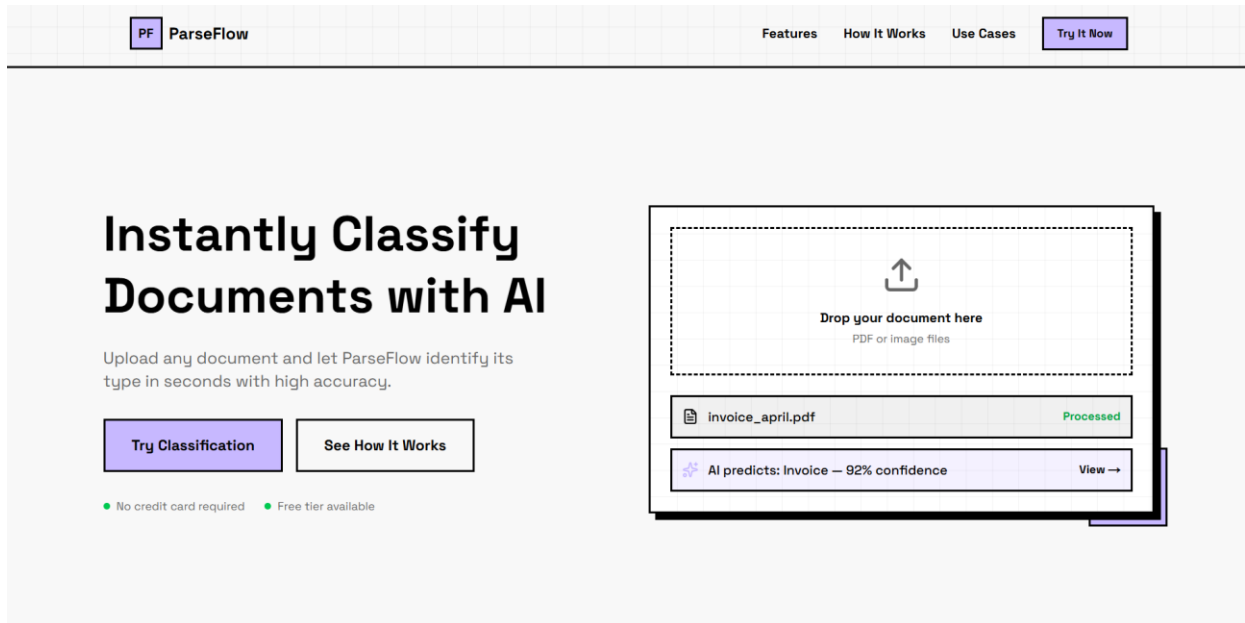


Fig 13.1 Home Page

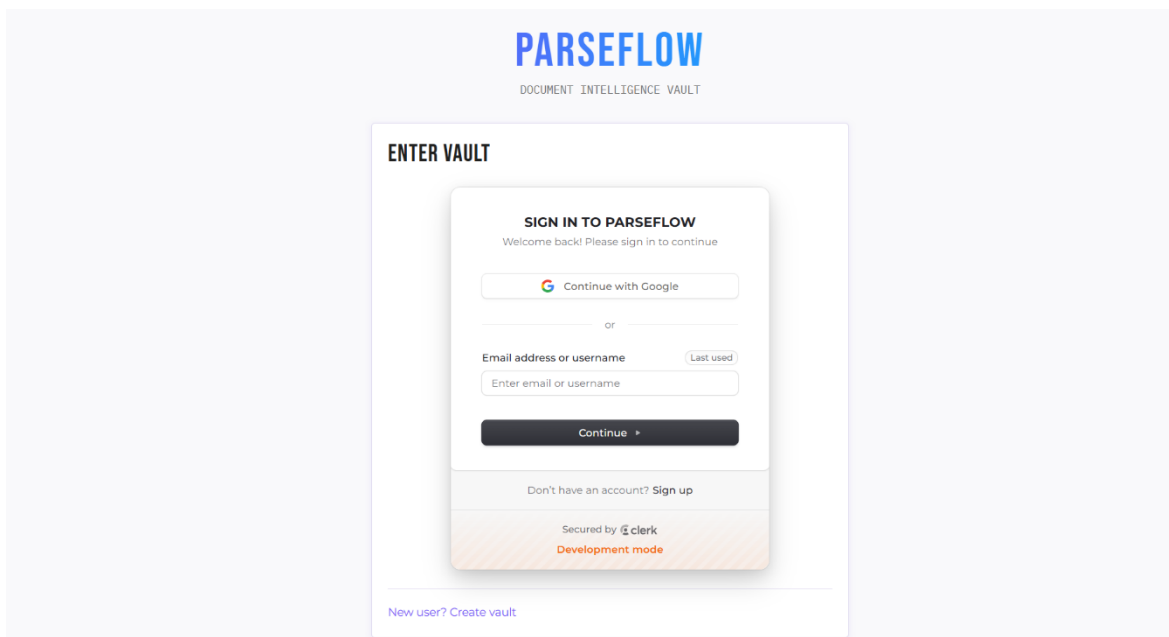


Fig 13.2 Login Page

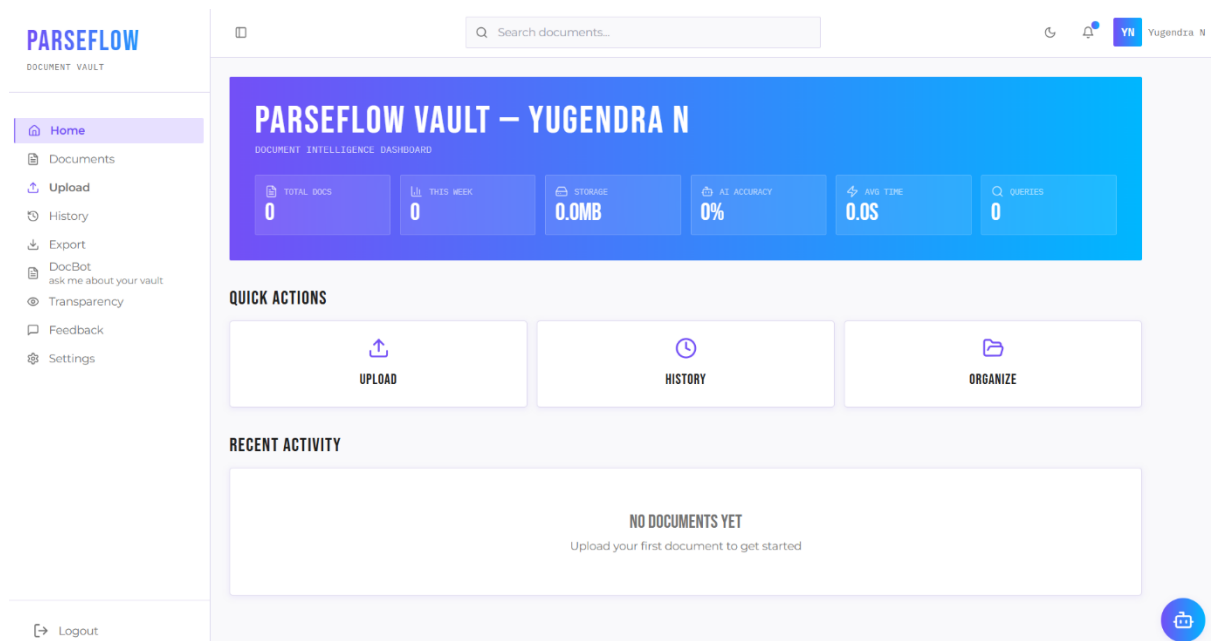


Fig 13.3 Dashboard

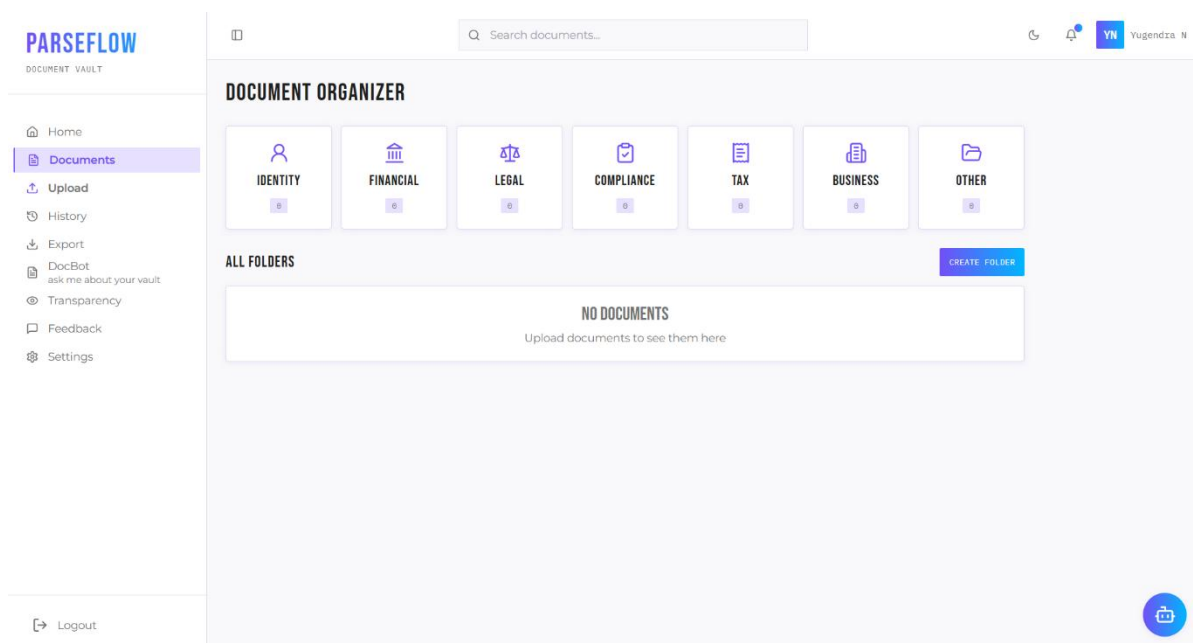


Fig 13.4 Organisation of documents

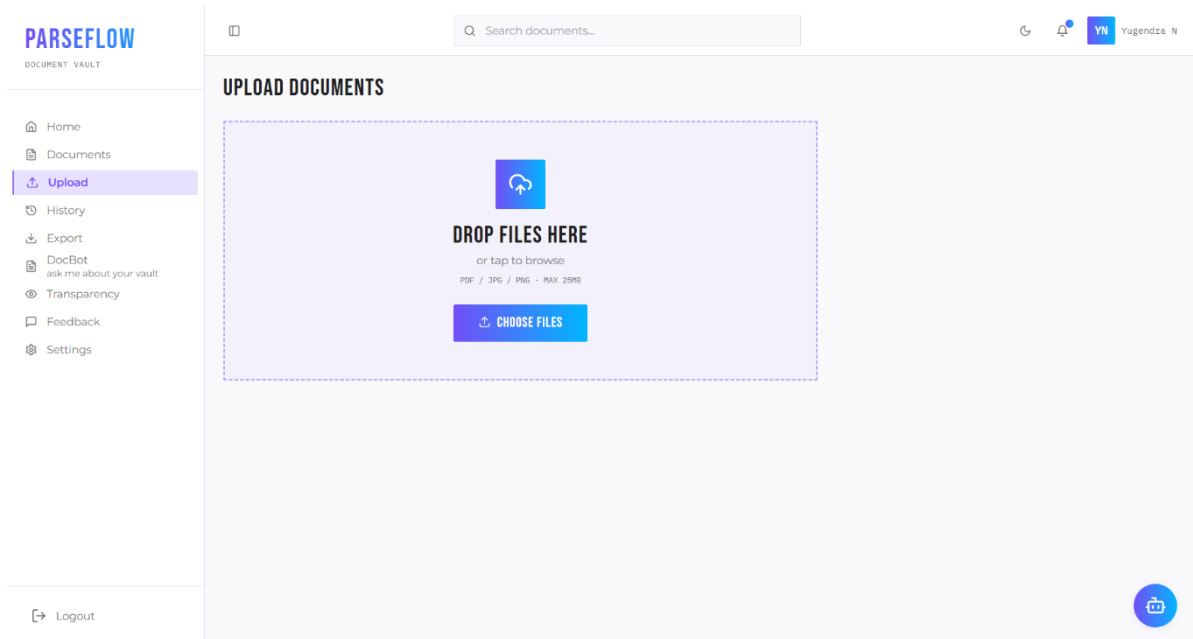


Fig 13.5 Upload document

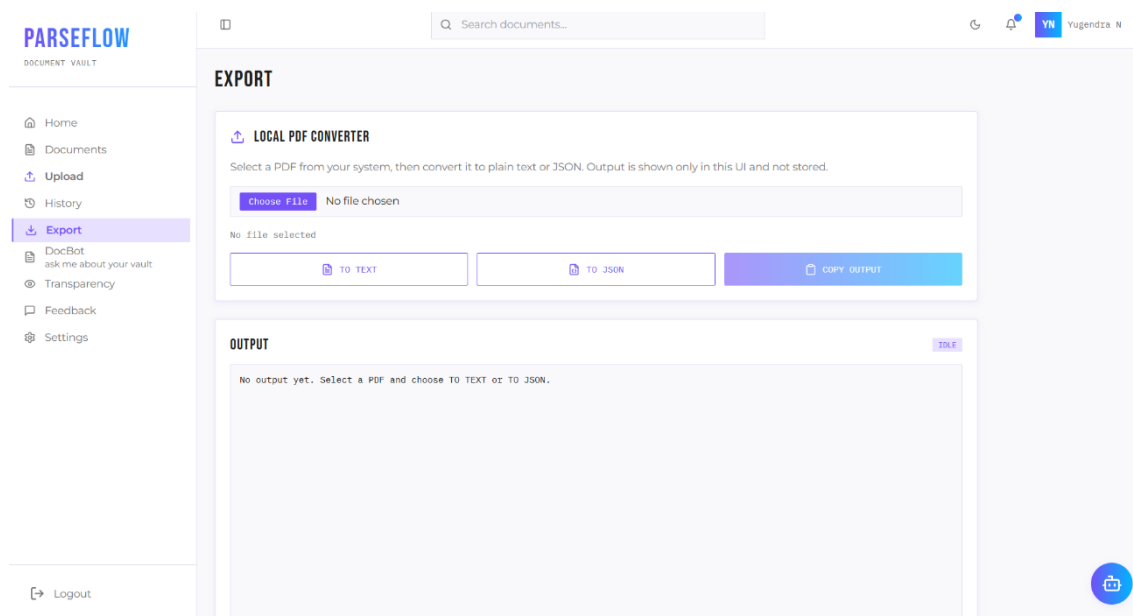


Fig 13.6 Export

These screenshots provide visual evidence of system implementation and demonstrate the usability and responsiveness of the application.



Sri Eshwar
College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai



PROJECT TITLE	PARSEFLOW – AI BASED DOCUMENT CLASSIFIER AND ORGANISER	
PROGRAM	B.TECH ARTIFICIAL INTELLIGENCE & DATA SCIENCE	
PROJECT BATCH NUMBER		
BATCH MEMBERS	722824243255	YUGENDRA N
NAME OF THE SUPERVISOR	Mr. S. Sam Peter M.E., (Ph.D.) Assistant Professor, AI&DS	
NAME OF THE SDG GOALS MAPPED	Decent work and economic growth, Industry , Innovation and Infrastructure	
MENTION THE SDG GOALS NUMBER	SDG 8 ,SDG 9	
NAME OF THE TRL LEVEL	Technology Demonstrated in Relevant Environment	
MENTION THE TRL LEVEL	TRL 6	

POs & PSOs Mapping (Put a tick mark in the mapped PO's & PSO's):

Program Outcomes											Program Specific Outcomes		
PO1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
✓	✓	✓	✓	✓		✓		✓	✓	✓	✓	✓	

Signature of the Supervisor
(Mr. S. Sam Peter)



VENUE AND EXPENDITURE STATEMENT FOR THE PROJECT WORK

Laboratory details where the project is carried out	DVA Lab, AI & DS
Software details	• Operating System: Windows • Node.js: 18.x or above • Python: 3.10 • React.js: 18.x • Express.js: 4.x • Code Editor: Visual Studio Code

Details of the Software

S. No	Software Components	Version
1.	Node.js	18.x+
2.	React.js	18.x+
3.	Express.js	4.x+
4.	MongoDB	14.x+
5.	Pg (Node MongoDB Driver)	8.x+
6.	Python	3.10
7.	Socket.io	4.x+
8.	Tailwind CSS	3.x+
9.	JWT (json web token)	9.x+
10.	Visual Studio Code	Latest
11.	Git & GitHub	Latest
12.	Chrome Browser	120.x+

Signature of the student

(YUGENDRA N)

Signature of the Supervisor

(Mr. S. Sam Peter)